MIT ACADEMY OF ENGINEERING, PUNE

Department of Computer Engineering

PROJECT REPORT

Jenkins CI/CD Pipeline with Terraform

Infrastructure as Code on AWS and Docker

Submitted by

Harshada Gahininath Gujar

B.Tech Computer Engineering

PRN / Roll No.: [fill in your PRN]

Course project: Jenkins CI/CD + Terraform Mini Project

Academic year 2026-27

Course instructor: [add instructor name]

GitHub repository: https://github.com/Harshadagujar2005/jenkins-terraform-mini-project

Demo video: [paste video link]

Abstract

Modern software teams ship small changes often, and doing the build, test and release steps by hand is slow and error prone. This project designs and implements a complete Continuous Integration and Continuous Delivery (CI/CD) workflow for a small Node.js web application, using Jenkins as the automation server, Docker for packaging and Terraform for Infrastructure as Code (IaC).

A single `Jenkinsfile` stored in the GitHub repository defines the pipeline. On every build Jenkins checks out the code, installs dependencies, runs automated tests, builds a Docker image tagged with the build number, validates and plans the Terraform configuration for AWS, deploys the application container locally with Terraform, and finally verifies the running application through a `/health` endpoint. An optional pipeline parameter (`APPLY_AWS`) lets the same pipeline create real cloud resources in AWS region ap-south-1 (Mumbai): an S3 bucket with versioning, a security group, and an EC2 `t2.micro` instance.

The pipeline was executed end to end, including a real `terraform apply` on AWS, and a second run after a code change showed the new version being rebuilt and redeployed automatically. The report describes the design, the implementation, the results with screenshots, the problems encountered and how they were solved.

Keywords: Jenkins, CI/CD, Terraform, Infrastructure as Code, Docker, AWS, DevOps, pipeline as code.

Table of Contents

1. Introduction.....4

2. Technology Overview.....5

3. System Design.....7

4. Implementation.....9

5. Execution and Results.....15

6. Testing and Verification.....21

7. Challenges and Solutions.....22

8. Security and Best Practices.....23

9. Advantages, Limitations and Future Scope.....24

10. Conclusion.....25

References.....25

Appendix A: Jenkinsfile.....27

Appendix B: Terraform Configuration.....29

Appendix C: Steps to Reproduce the Project.....33

1. Introduction

1.1 Background

Software development has moved from large, infrequent releases to small changes that are released many times a day. To make this safe, teams automate the steps between writing code and running it for users. This practice is called **DevOps**, and two of its core ideas are **Continuous Integration (CI)** and **Continuous Delivery (CD)**.

In Continuous Integration, every change a developer pushes is automatically built and tested, so that integration problems are found within minutes instead of weeks. In Continuous Delivery, a change that passes the automated checks is packaged and deployed in a repeatable way. Jenkins is one of the most widely used open-source automation servers for building such pipelines.

Alongside automation of the application lifecycle, teams also automate the infrastructure the application runs on. **Infrastructure as Code (IaC)** describes servers, networks and storage in text files that are version controlled, reviewed and executed by a tool. Terraform, from HashiCorp, is a popular IaC tool that works with many providers, including AWS and Docker.

1.2 Problem Statement

Building, testing and deploying an application by hand has several problems: steps get forgotten, different people do them differently, mistakes reach production, and nobody can easily reproduce what was done. Creating cloud resources through a web console has the same weakness: the result cannot be reviewed, repeated or rolled back easily.

The problem addressed in this project is to design a small but complete workflow in which a single push of source code triggers an automated, observable and repeatable sequence of build, test, infrastructure and deployment steps.

1.3 Objectives

- Create a Jenkins pipeline job connected to a GitHub source-code repository.
- Automate checkout, dependency installation, testing and building of a Node.js application.
- Package the application as a Docker image and deploy it automatically.
- Write Terraform configuration for AWS (S3, security group, EC2) and for a local Docker deployment.
- Execute `terraform init`, `plan` and `apply` from the pipeline, including a real apply on AWS.
- Demonstrate CI/CD in action by changing the code and observing an automatic redeployment.
- Explain Infrastructure as Code and the pipeline design clearly in a report and a recorded demonstration.

1.4 Scope

The application is intentionally small (one Express server with a home page and a health endpoint), because the focus of the project is the delivery pipeline and not the application. Jenkins runs in a Docker container on the developer's own computer. The application is deployed to a local Docker container using Terraform, and a separate Terraform configuration provisions real infrastructure on AWS. Production concerns such as load balancing, monitoring, multiple environments and remote Terraform state are discussed under future scope (Section 9).

1.5 Organisation of the Report

Section 2 introduces the tools. Section 3 presents the architecture and pipeline design. Section 4 explains the implementation file by file. Section 5 shows the execution results with screenshots, Section 6 covers testing, Section 7 lists the challenges faced, Section 8 discusses security, Section 9 gives advantages, limitations and future scope, and Section 10 concludes. The appendices contain the complete Jenkinsfile, the Terraform files and the reproduction steps.

2. Technology Overview

Table 1: Technologies used

Technology	Version / detail	Role in this project
Node.js and Express	Node 20, Express 4	Sample web application with `/` and `/health` routes
Git and GitHub	Public repository	Source control and the repository Jenkins pulls from
Jenkins	2.541.3 (LTS, JDK 17) in Docker	Automation server that runs the pipeline
Docker	Docker Desktop, socket mounted into Jenkins	Packages the app as an image and runs the container
Terraform	1.9.8	Infrastructure as Code for AWS and Docker
Terraform AWS provider	hashicorp/aws 5.100.0	Creates S3, security group and EC2
Terraform Docker provider	kreuzwerker/docker ~> 3.0	Creates and runs the application container
AWS	Region ap-south-1 (Mumbai)	Cloud platform for the real infrastructure

2.1 Continuous Integration and Continuous Delivery

A CI/CD pipeline is an automated sequence of stages that moves a code change from the repository to a running system. A typical pipeline contains the following stages, all of which are present in this project:

- **Source:** fetch the latest code from the repository.
- **Build:** install dependencies and produce a deployable artifact (here, a Docker image).

- **Test:** run automated tests; a failure stops the pipeline so that broken code is never deployed.
- **Provision:** create or update the infrastructure the application needs (Terraform).
- **Deploy:** run the new version of the application.
- **Verify:** check that the deployed application is healthy.

2.2 Jenkins and Pipeline as Code

Jenkins is an open-source automation server with a large plugin ecosystem. Work is described as a **Pipeline**, which can be written in a text file called a `Jenkinsfile` using a declarative syntax. Keeping the Jenkinsfile in the same repository as the application is called "pipeline as code": the delivery process is versioned, reviewed and changed together with the code. In this project the Jenkins job uses the option **Pipeline script from SCM**, so Jenkins reads the Jenkinsfile from GitHub on every build.

Jenkins stores secrets such as cloud keys in its **Credentials** store. A pipeline can use a stored credential inside a `withCredentials` block, which injects the values as environment variables and masks them in the console output. This is how the AWS keys are used in this project without ever being written to Git.

2.3 Docker

Docker packages an application and everything it needs into an image, and runs the image as an isolated container. The same image behaves the same on any machine, which removes the "it works on my machine" problem. In this project Docker is used in three ways: the application image is built from a `Dockerfile`, the application is run as a container, and Jenkins itself runs as a container. The Jenkins container is given access to the host's Docker daemon by mounting the Docker socket, so pipeline steps can build images and Terraform can create containers on the host.

2.4 Terraform and Infrastructure as Code

Terraform is an IaC tool. The engineer writes **declarative** configuration in HashiCorp Configuration Language (HCL) describing the resources that should exist. Terraform compares that desired state with what exists, and works out the changes needed. The main commands are shown in Table 2.

Table 2: Core Terraform commands

Command	What it does
`terraform init`	Downloads the required providers and prepares the working directory
`terraform fmt`	Rewrites the files into the standard canonical format
`terraform validate`	Checks that the configuration is syntactically and internally valid
`terraform plan`	Shows what Terraform would create, change or destroy, without doing it
`terraform apply`	Creates or updates the real resources to match the configuration
`terraform destroy`	Removes all the resources managed by the configuration

Terraform records the resources it manages in a **state file**. The state lets Terraform know which real resource corresponds to which block of configuration, so later runs update or delete the right objects. The benefits of IaC are repeatability, version control, peer review, the ability to recreate an environment quickly, and documentation of the infrastructure in the code itself.

2.5 Amazon Web Services

AWS is a cloud platform. This project uses three services: **S3** (object storage, used here as a bucket for build artifacts with versioning enabled), **EC2** (virtual servers, a `t2.micro` instance which is in the AWS free tier) and **security groups** (virtual firewalls controlling traffic to the instance). Access is through an **IAM** user with the managed policies `AmazonS3FullAccess` and `AmazonEC2FullAccess`, using an access key pair.

3. System Design

3.1 Requirements

Table 3: Functional and non-functional requirements

Type	Requirement
Functional	Jenkins fetches the code from GitHub on demand and runs the Jenkinsfile
Functional	Tests must run automatically and a failing test must stop the pipeline
Functional	The application must be packaged as a Docker image and deployed to a container
Functional	Terraform must provision AWS resources, with an option to run plan only
Functional	The deployment must be verified automatically through a health check
Non-functional	Secrets (AWS keys) must never be stored in the repository
Non-functional	The setup must be reproducible by anyone using Docker and the repository
Non-functional	AWS resources must stay in the free tier and be removable with one command

3.2 Overall Architecture

Figure 1 shows the architecture. The developer pushes code to GitHub. Jenkins, running as a container on the local machine, pulls the repository and runs the pipeline. Through the mounted Docker socket the pipeline builds the application image and Terraform creates the application container on the same machine. In parallel, Terraform talks to AWS with the credential stored in Jenkins and provisions the cloud resources in the Mumbai region.

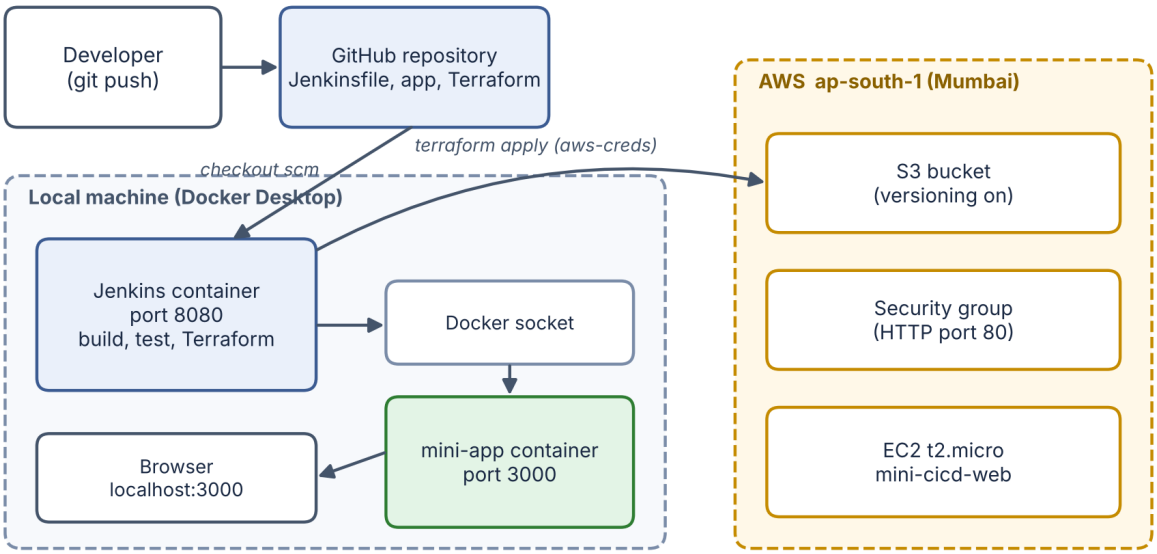


Figure 1: Overall architecture of the CI/CD and IaC workflow

3.3 Pipeline Design

The pipeline has eleven stages, shown in Figure 2. The order is deliberate: tests run before anything is built so that bad code fails fast; the Docker image is built before Terraform deploys it; the AWS plan runs before the optional apply so the changes can be reviewed; and the health check is the last stage so that a successful pipeline means a working application.

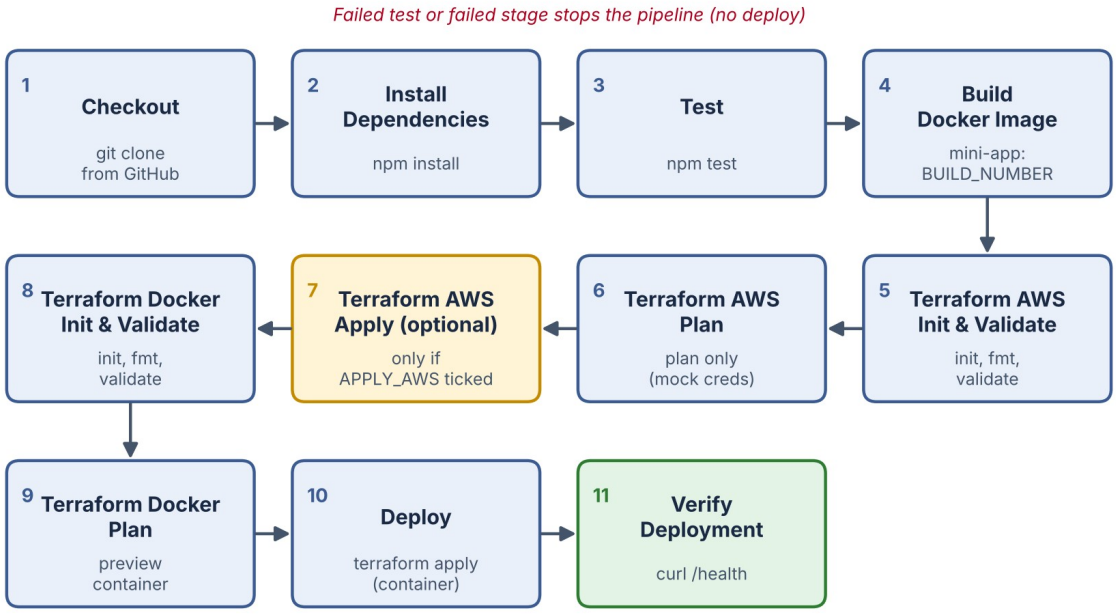


Figure 2: Stages of the Jenkins pipeline (read left to right, then follow the arrows)

Two Jenkins options make the pipeline safer to run: `timestamps()` adds a time to every console line, which helps with debugging, and `disableConcurrentBuilds()` prevents two builds from changing the same infrastructure at the same time.

3.4 Two Terraform Configurations

The infrastructure is split into two independent Terraform directories, because they have different needs:

- **`terraform/docker`** deploys the application container on the local machine. It needs no cloud account, so every build can run a real `apply` and the CI/CD result is always visible at `http://localhost:3000`.
- **`terraform/aws`** provisions cloud infrastructure. Because it can create billable resources, its `apply` is protected by the `APPLY_AWS` build parameter. With the parameter off, the stage runs only `plan`.

A feature of the AWS configuration is the variable `use_mock_credentials`. When it is `true` (the default), the provider is configured with dummy keys and the skip flags, so `init`, `validate` and `plan` work with no AWS account. When it is `false`, the provider uses the real credentials injected by Jenkins. This lets the same code be tested safely and applied for real.

3.5 Design Decisions

Table 4: Key design decisions

Decision	Reason
Jenkins in a Docker container	Same setup on any computer; one `docker compose up` replaces a manual installation
Mount the Docker socket	Lets pipeline steps and Terraform use the host Docker daemon without Docker-in-Docker
Pipeline script from SCM	The Jenkinsfile is versioned together with the code
Image tag = build number	Every deployment is traceable to a build and old images can be rolled back to
Reusable Terraform module	The container definition is separated (`app_container`) and can be reused for other apps
`APPLY_AWS` parameter	Prevents accidental creation of billable cloud resources
Health-check stage	A pipeline is only "green" if the application actually responds

4. Implementation

The project repository has the following structure:

```
jenkins-terraform-mini-project/
|-- Jenkinsfile                  # pipeline definition
|-- Dockerfile                   # image of the application
|-- server.js, package.json      # Node.js / Express application
|-- test/app.test.js             # automated tests
|-- jenkins/
|   |-- Dockerfile               # Jenkins image with Docker CLI, Node, Terraform
|   `-- docker-compose.yml       # starts Jenkins
|-- terraform/
|   |-- aws/                     # S3, security group, EC2
|   `-- docker/                  # container deployment + reusable module
`-- docs/                        # report, screenshots, demo guide
```

4.1 The Application

The application is a small Express server. It exposes two routes: `/` returns an HTML page showing a greeting and the environment name, and `/health` returns a JSON document `{"status":"ok"}`. The server exports the Express app so that the tests can start it on a random port. The port and environment label are read from environment variables, which Terraform sets when it creates the container.

After the code-change step of the demo, the home page was upgraded to a styled card that shows a "Version 2 - deployed automatically" badge, which makes the redeployment visible in the browser.

4.2 Dockerfile of the Application

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY server.js ./
EXPOSE 3000
CMD ["node", "server.js"]
```

The image is based on the small `node:20-alpine` image. Copying `package*.json` first and installing dependencies before copying the source lets Docker cache the dependency layer, so rebuilds after a code-only change are fast. Development dependencies and tests are not included in the production image.

4.3 Running Jenkins in Docker

Jenkins is started with `docker compose up -d --build` from the `jenkins` folder. The compose file builds a custom image, publishes port 8080 (web interface) and 50000 (agents), stores all Jenkins data in a named volume `jenkins_home`, and mounts `/var/run/docker.sock` so that Jenkins can control the host's Docker. The line `extra_hosts: host.docker.internal:host-gateway` lets the Jenkins container reach the application published on the host, which the verification stage relies on.

```
services:
  jenkins:
    build: .
    container_name: jenkins
    user: root
    ports: ["8080:8080", "50000:50000"]
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    extra_hosts:
      - "host.docker.internal:host-gateway"
volumes:
  jenkins_home:
```

The custom Jenkins image (`jenkins/Dockerfile`) starts from `jenkins/jenkins:lts-jdk17` and adds the Docker command-line client, `curl`, `unzip`, Node.js 20 and Terraform 1.9.8, so that every tool the pipeline calls is available inside the Jenkins container. A `docker --version` check at the end of the build makes the image build fail early if the Docker client is missing.

```
PS C:\Users\Prime\Downloads\jenkins-terraform-mini-project\jenkins-terraform-mini-project>
cd jenkins
PS C:\Users\Prime\Downloads\jenkins-terraform-mini-project\jenkins-terraform-mini-project\j
enkins> docker compose up -d --build
[+] Building 6.7s (10/10) FINISHED
 => [internal] load local bake definitions
                          0.0s
 => => reading from stdin 657B
                          0.0s
 => [internal] load build definition from Dockerfile
                          0.1s
 => => transferring dockerfile: 806B
```

Figure 3: Starting Jenkins with docker compose up -d --build

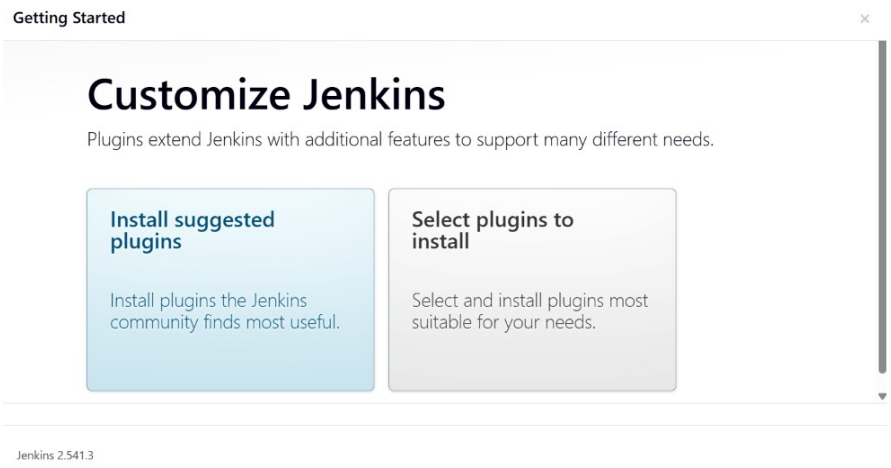


Figure 4: Jenkins first-time setup: choosing the plugins

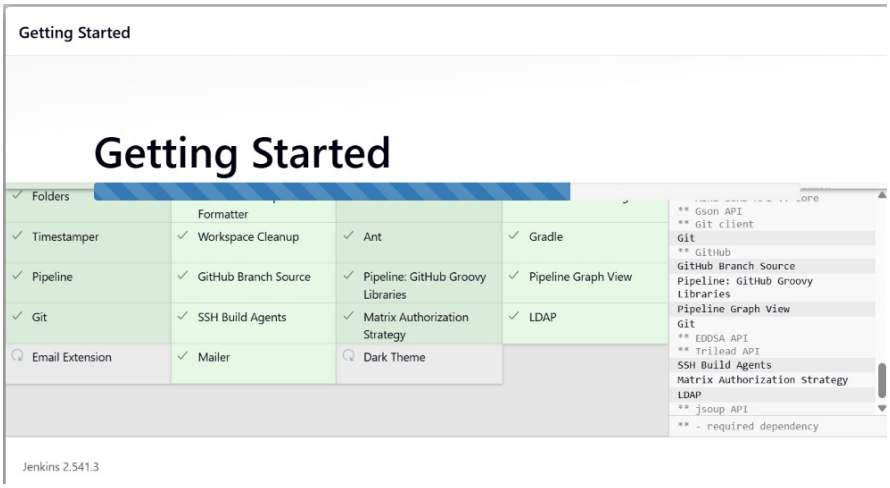


Figure 5: Installation of the suggested plugins (Git, Pipeline, Timestamper and others)

4.4 Credentials and Job Configuration

The AWS access key pair was created for a dedicated IAM user and stored in Jenkins as a credential of the kind "Username with password" with the ID `aws-creds` (username = access key ID, password = secret access key). The ID must match the one used in the Jenkinsfile. The credential never appears in the repository.

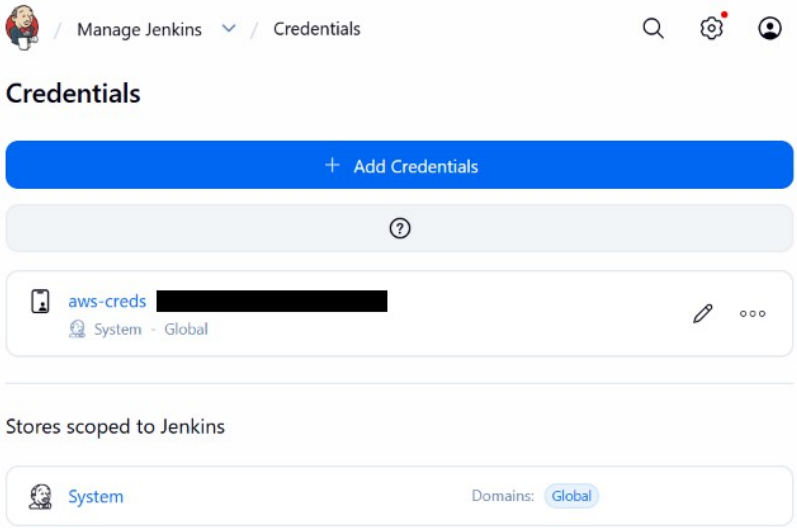


Figure 6: The aws-creds credential in the Jenkins credentials store (key ID hidden)

The pipeline job `mini-cicd-pipeline` is of type Pipeline, with the definition **Pipeline script from SCM**, SCM = Git, the GitHub repository URL, branch `*/main` and script path `Jenkinsfile` (Figure 8 in Section 5).

4.5 The Jenkinsfile

The complete Jenkinsfile is in Appendix A. This section explains each part.

4.5.1 Environment, parameters and options

```
environment {
    IMAGE_NAME = 'mini-app'
    IMAGE_TAG  = "${env.BUILD_NUMBER}"
    APP_PORT   = '3000'
}
parameters {
    booleanParam(name: 'APPLY_AWS', defaultValue: false, ...)
}
options { timestamps(); disableConcurrentBuilds() }
```

The image tag is the Jenkins build number, so build 4 produces `mini-app:4`. The boolean parameter `APPLY_AWS` is false by default, so a normal build never creates AWS resources.

4.5.2 Build and test stages

- **Checkout:** `checkout scm` clones the repository configured in the job.
- **Install Dependencies:** prints the Node and npm versions and runs `npm install`.
- **Test:** runs `npm test`. If any test fails, the shell returns a non-zero exit code and Jenkins stops the pipeline, so nothing broken is deployed.
- **Build Docker Image:** runs `docker build -t mini-app:<build number> .`

4.5.3 Terraform AWS stages

```
stage('Terraform AWS: Plan') {
  steps {
    dir('terraform/aws') {
      sh 'terraform plan -input=false -no-color -var="use_mock_credentials=true" | tee
aws-plan.txt'
    }
    archiveArtifacts artifacts: 'terraform/aws/aws-plan.txt'
  }
}
stage('Terraform AWS: Apply (optional)') {
  when { expression { return params.APPLY_AWS == true } }
  steps {
    withCredentials([usernamePassword(credentialsId: 'aws-creds',
        usernameVariable: 'AWS_ACCESS_KEY_ID', passwordVariable:
'AWS_SECRET_ACCESS_KEY')]) {
      dir('terraform/aws') {
        sh 'terraform apply -auto-approve -input=false -
var="use_mock_credentials=false"'
      }
    }
  }
}
```

The plan is saved as a build artifact so it can be reviewed later. The apply stage has a `when` condition, so it is skipped unless `APPLY_AWS` is ticked. When it runs, `withCredentials` exposes the stored key pair as the standard AWS environment variables for that block only.

4.5.4 Docker deployment and verification

The Terraform Docker stages initialise, validate, plan and apply the configuration in `terraform/docker`, passing the image name, tag and port as variables. Terraform replaces the previous container with one created from the new image. The final stage waits for the application to answer:

```
for i in 1 2 3 4 5 6 7 8 9 10; do
  if curl -fsS http://host.docker.internal:${APP_PORT}/health; then
    echo; echo "Deployment verified."; exit 0
  fi
  echo "Waiting for app..."; sleep 3
done
echo "App did not become healthy"; exit 1
```

The loop tries up to ten times, three seconds apart, because the container needs a moment to start. If the health endpoint never answers, the stage fails and so does the pipeline.

4.6 Terraform Configuration for AWS

The AWS configuration (`terraform/aws/main.tf`) declares four managed resources, which is why the apply reports four resources added:

Table 5: Resources created in AWS

Resource	Purpose
`aws_s3_bucket.artifacts`	Bucket for build artifacts (name set by `bucket_name`)
`aws_s3_bucket_versioning.artifacts`	Turns on versioning so old object versions are kept
`aws_security_group.web`	Allows inbound HTTP (port 80) and all outbound traffic
`aws_instance.web`	`t2.micro` EC2 instance; `user_data` installs Docker and starts an nginx container

```
resource "aws_s3_bucket" "artifacts" {
  bucket = var.bucket_name
  tags = { Name = "${var.project_name}-artifacts", Project = var.project_name }
}
resource "aws_s3_bucket_versioning" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id
  versioning_configuration { status = "Enabled" }
}
resource "aws_instance" "web" {
  ami                    = var.ami_id
  instance_type          = var.instance_type
  vpc_security_group_ids = [aws_security_group.web.id]
  tags = { Name = "${var.project_name}-web", Project = var.project_name }
}
```

All values that may change are variables (`aws_region`, `project_name`, `bucket_name`, `ami_id`, `instance_type`, `use_mock_credentials`) with defaults in `variables.tf`, and `outputs.tf` prints the bucket name, the security group ID and the instance type. This separation of `main.tf`, `variables.tf` and `outputs.tf` is the standard Terraform layout and makes the configuration reusable: another person only changes the variables, not the resources. The `aws_s3_bucket.artifacts.id` reference also shows an **implicit dependency**: Terraform creates the bucket first, then applies the versioning to it.

4.7 Terraform Configuration for Docker and the Reusable Module

The Docker configuration calls a local module named `app_container`. The root file `main.tf` configures the Docker provider and passes five values into the module; the module contains the actual `docker_container` resource.

```
# terraform/docker/main.tf
```

```
module "app" {
  source         = "./modules/app_container"
  container_name = var.container_name
  image          = "${var.image_name}:${var.image_tag}"
  internal_port  = 3000
  external_port  = var.external_port
  environment    = var.environment
}

# terraform/docker/modules/app_container/main.tf
resource "docker_container" "app" {
  name    = var.container_name
  image   = var.image
  restart = "unless-stopped"
  ports { internal = var.internal_port  external = var.external_port }
  env = ["APP_ENV=${var.environment}", "PORT=${var.internal_port}"]
}
```

Because the module takes the container name, image and ports as inputs, the same module could deploy a second application without any change. When the image tag changes (a new build number), Terraform sees that the container must be replaced and performs the update, which is exactly the redeploy seen in the demonstration.

5. Execution and Results

The pipeline was run four times. Table 6 summarises the runs; the sections after it show the evidence.

Table 6: Summary of pipeline runs

Build	Purpose	Result
#1	First run	Failed at Build Docker Image: `docker: not found` (Docker CLI missing in the Jenkins image)
#2	Run after the fix, `APPLY_AWS` unticked	Success: tests passed, image built, AWS plan only, container deployed, health check passed
#3	`APPLY_AWS` ticked	Success: S3 bucket and EC2 instance created in AWS (real apply)
#4	Run after changing the code on GitHub	Success: image `mini-app:4` built, container replaced, new page visible

5.1 Repository and Job Configuration

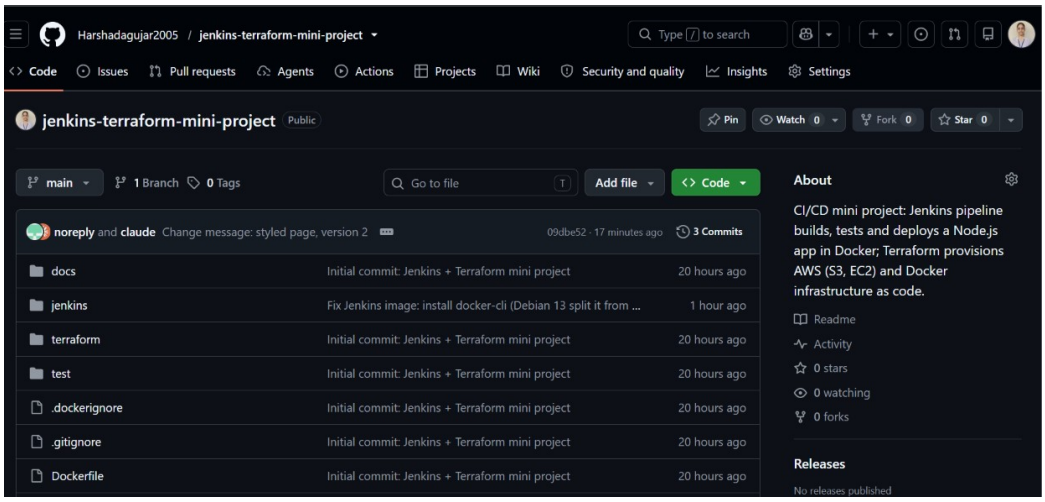


Figure 7: The public GitHub repository with the Jenkinsfile, Dockerfile, terraform and test folders

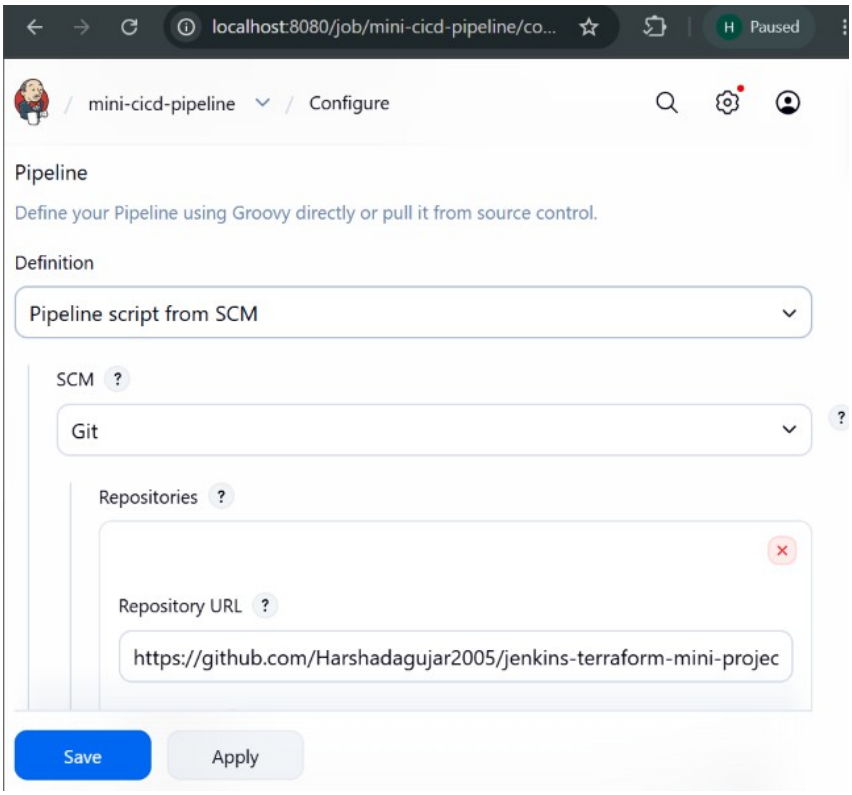


Figure 8: Jenkins job configuration: Pipeline script from SCM, Git and the repository URL



Figure 9: Console output of the first stage: Jenkins obtains the Jenkinsfile from GitHub and clones the repository

5.2 The First Successful Run (build #2)

After the Docker CLI fix, build #2 ran with `APPLY_AWS` unticked. Figure 10 shows the pipeline while Terraform downloads the large AWS provider during `terraform init`. This step took several minutes on the first run only. The console output at the end of the run shows the health check and the final result.

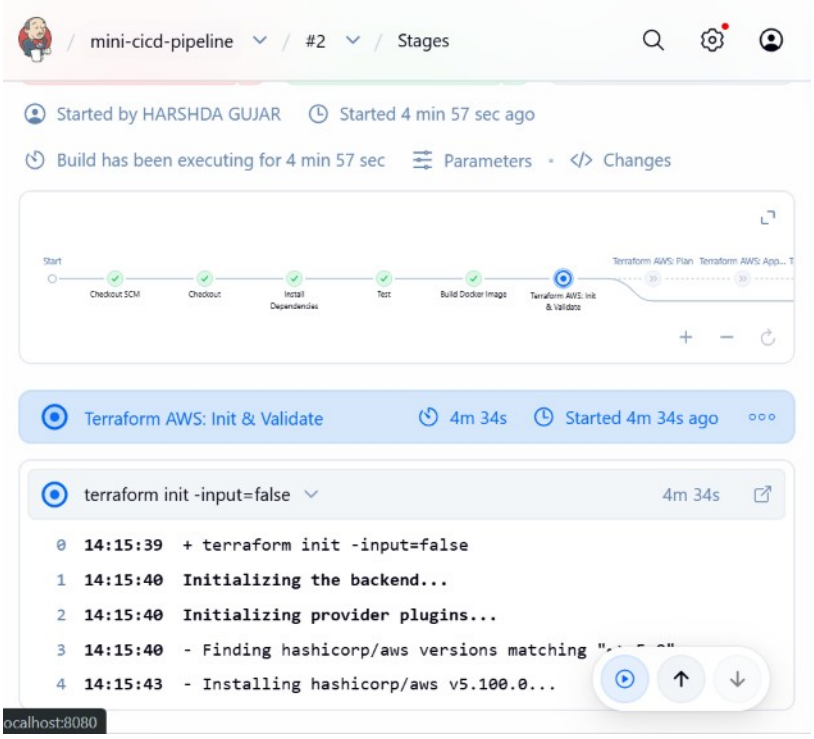


Figure 10: Build #2 in progress: all earlier stages are green and Terraform AWS init is downloading the provider

```
14:26:10  + echo Deployment verified.
14:26:10  Deployment verified.
14:26:10  + exit 0
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
14:26:10  Pipeline succeeded. App is running at http://localhost:3000
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 11: End of the console output of build #2: "Deployment verified." and "Finished: SUCCESS"

5.3 Real AWS Deployment (build #3)

For build #3 the parameter `APPLY_AWS` was ticked, which made the Terraform AWS Apply stage run with the `aws-creds` credential. Terraform created the S3 bucket with versioning, the security group and the EC2 instance. All stages were green.

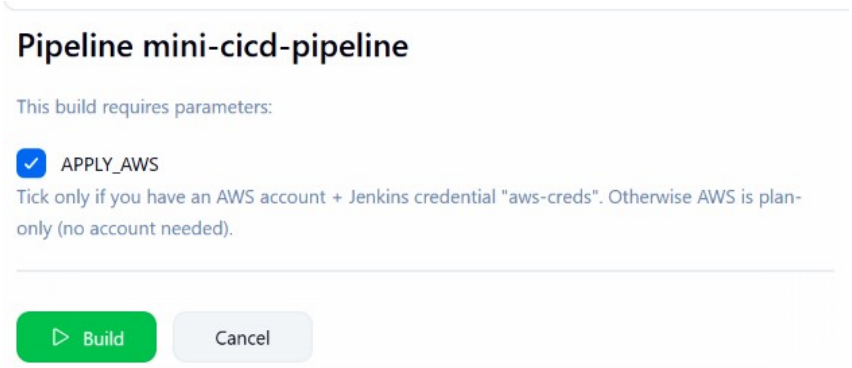


Figure 12: Build with Parameters: APPLY_AWS ticked for the real AWS apply

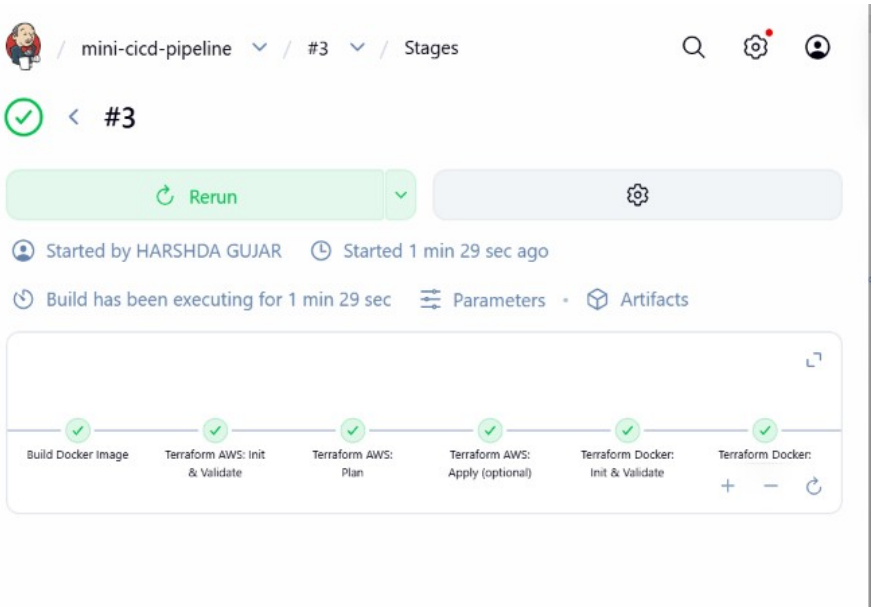


Figure 13: Stage view of build #3: all stages green including Terraform AWS: Apply (optional)

The resources were then confirmed in the AWS console in the Asia Pacific (Mumbai) region: the instance `mini-cicd-web` was running with 2/2 status checks passed, and the bucket `mini-cicd-project-harshada` existed in ap-south-1.

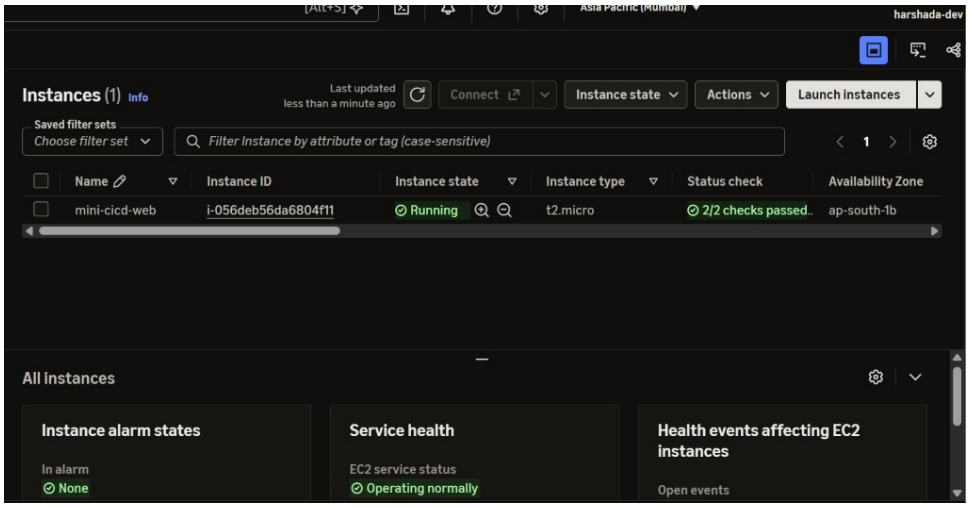


Figure 14: AWS console (EC2): instance mini-cicd-web, type t2.micro, running, 2/2 checks passed

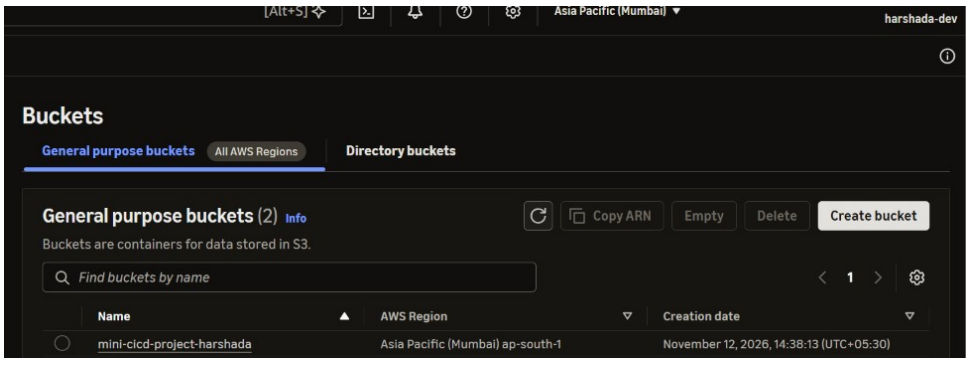


Figure 15: AWS console (S3): bucket mini-cicd-project-harshada in Asia Pacific (Mumbai)

5.4 Code Change and Automatic Redeployment (build #4)

To demonstrate CI/CD, the home page in `server.js` was changed and pushed to GitHub (commit message "Change message"). A new build was started, Jenkins fetched the new commit, ran the tests again, built `mini-app:4` and Terraform replaced the running container. Refreshing `http://localhost:3000` then showed the new styled page with the "Version 2" badge. The output of `docker ps` shows the container created from the new image, next to the Jenkins container.

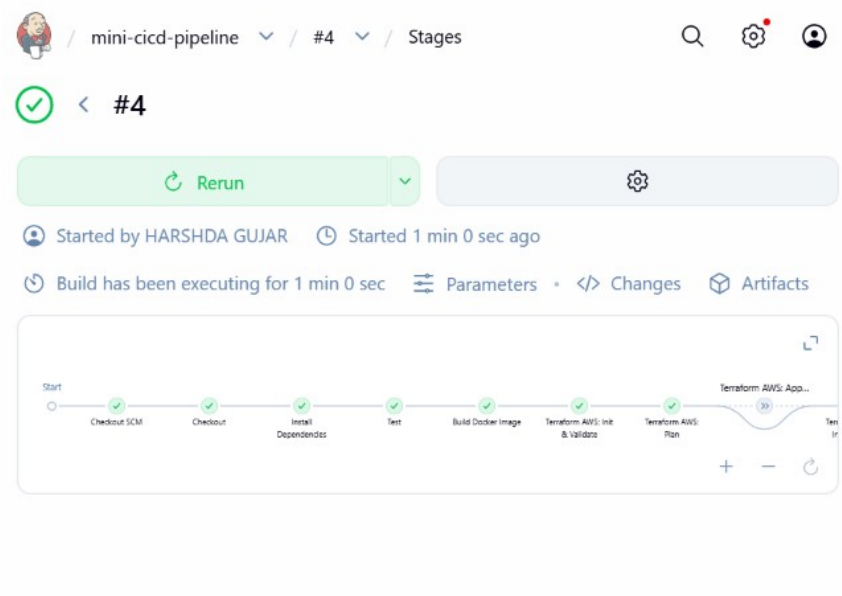


Figure 16: Stage view of build #4 after the code change

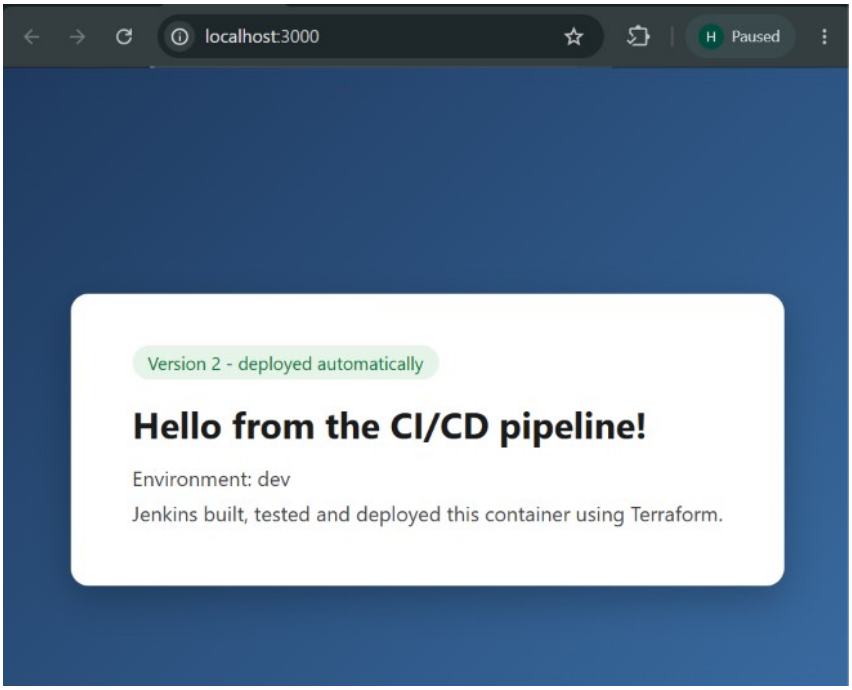


Figure 17: The deployed application at http://localhost:3000 after the redeploy

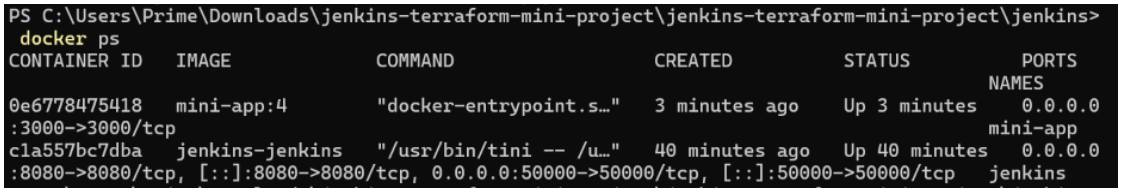


Figure 18: docker ps: the mini-app:4 container and the Jenkins container

5.5 Results Summary

- The pipeline runs unattended from checkout to a verified deployment in roughly one to two minutes once the Terraform providers are cached.
- The two automated tests pass in under one second and guard every deployment.
- Terraform created four AWS resources from code and the same code can remove them with `terraform destroy`.
- A change pushed to GitHub reached the running application without any manual build or deploy step.

6. Testing and Verification

6.1 Automated Unit and Integration Tests

The tests use the test runner built into Node.js (`node --test`), so no extra test framework is needed. The test file starts the Express app on a random free port, sends real HTTP requests to it, and checks the responses:

```
test('GET /health returns status ok', async () => {
  const res = await fetch(`${base}/health`);
  assert.strictEqual(res.status, 200);
  const body = await res.json();
  assert.strictEqual(body.status, 'ok');
});

test('GET / returns the hello page', async () => {
  const res = await fetch(`${base}/`);
  assert.strictEqual(res.status, 200);
  const text = await res.text();
  assert.ok(text.includes('Hello from the CI/CD pipeline'));
});
```

Both tests passed in every run (2 tests, 2 passed, 0 failed). Because the Test stage comes before the image build, a failing test would stop the pipeline before anything is built or deployed.

6.2 Static Checks on Infrastructure Code

Before planning, the pipeline runs `terraform fmt -recursive`, which keeps the files in canonical style, and `terraform validate`, which detects syntax errors, unknown arguments and bad references without contacting any cloud service. This catches mistakes in the infrastructure code in the same way that unit tests catch mistakes in the application code.

6.3 Plan Before Apply

`terraform plan` previews the exact resources that would be created, changed or destroyed. In this project the AWS plan runs on every build (using mock credentials, so it needs no account) and is archived as `aws-plan.txt`. The real apply happens only when a person explicitly ticks `APPLY_AWS`. This follows the good practice of reviewing a plan before changing real infrastructure.

6.4 Deployment Verification

The last stage calls `/health` on the deployed container through the host and fails the build if no answer arrives within about thirty seconds. This verifies the complete chain: image build, container start, port mapping and application start-up.

Table 7: Verification summary

Check	Stage	Result
Application tests (2)	Test	Passed
Docker image build	Build Docker Image	Passed (after Docker CLI fix)
Terraform format and validation	Terraform AWS / Docker: Init & Validate	Passed
AWS plan (4 resources to add)	Terraform AWS: Plan	Passed
AWS apply (real)	Terraform AWS: Apply (optional)	Passed, resources seen in the console
Container deployment	Deploy (Terraform Apply)	Passed
Health check	Verify Deployment	Passed ("Deployment verified.")

7. Challenges and Solutions

Several problems appeared while building and running the project. Solving them was an important part of the learning. Table 8 lists them, followed by details of the main ones.

Table 8: Problems and solutions

Problem	Cause	Solution
`docker: not found` in the Build Docker Image stage	The newer Debian-based Jenkins image ships the Docker CLI in a separate `docker-cli` package, so installing `docker.io` did not provide the `docker` command	Changed `jenkins/Dockerfile` to install `docker-cli` (with `docker.io` as a fallback), added a `docker --version` check, rebuilt the image
Could not sign in to Jenkins	The admin username and password were forgotten	Recreated Jenkins with `docker compose down -v` and set it up again; the credential and the job were added again
Jenkins asked for an "Administrator password"	The first-run setup wizard generates a one-time password	Read it with `docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword`
`terraform init` very slow on the first build	The AWS provider is about 700 MB to download and unpack	Waited for the download; the provider stays in the workspace so

		later builds are fast
Local folder was not a Git repository	The project had been unzipped, not cloned	Pushed the project to GitHub and used a fresh clone for later changes
Bucket name must be globally unique	S3 bucket names are shared by all AWS accounts	Used a project-specific name; documented the variable in `variables.tf`

7.1 Docker CLI Missing in Jenkins

This was the most instructive failure. Build #1 passed checkout, dependency installation and tests, and then stopped with exit code 127 and the message `docker: not found`. Exit code 127 means "command not found", which pointed to a missing tool and not to a bug in the pipeline. The Jenkins base image had been updated to a newer Debian release, where the package `docker.io` contains only the Docker daemon and the client moved into `docker-cli`. After adding that package and rebuilding the image, the command was available and the rest of the pipeline ran. The lesson is to pin versions and to add a check for every tool the pipeline needs.

7.2 Losing Access to Jenkins

Because all Jenkins configuration lives in the `jenkins_home` volume, deleting the volume resets Jenkins completely, including users, credentials and jobs. This confirmed how important it is to keep the pipeline definition in Git: the Jenkinsfile and all Terraform code were unaffected, and only the credential and the job configuration had to be recreated by hand. The reset took a few minutes, which is the practical benefit of pipeline as code and infrastructure as code.

7.3 Terraform Provider Download Time

The first `terraform init` of the AWS configuration appeared stuck for several minutes at "Installing hashicorp/aws v5.100.0". Checking the size of the `.terraform` directory inside the container showed that it kept growing (675 MB at the time of checking), so the process was working and only the network was slow. A way to speed up repeated runs would be a shared provider cache directory, listed in the future scope.

8. Security and Best Practices

- **No secrets in Git.** The AWS keys exist only in the Jenkins credential store. The repository contains the dummy values `mock-access-key` and `mock-secret-key` that are used for plan-only runs. A `terraform.tfvars.example` file is provided instead of a real variables file, and `.gitignore` excludes state files and local variable files.
- **Least-privilege thinking.** A separate IAM user (`jenkins-terraform`) is used instead of the account root user. The managed policies used (`AmazonS3FullAccess`, `AmazonEC2FullAccess`) are broader than necessary and are acceptable only for a learning project; in production a custom policy limited to the needed actions and resources should be used.

- **Protect billable actions.** The `APPLY_AWS` parameter defaults to false and a human must tick it, so a routine build cannot create cloud resources.
- **Clean up after use.** The AWS resources are destroyed with `terraform destroy` after the demonstration and the IAM access key is deactivated, to avoid charges and to remove the credential.
- **Credential masking.** `withCredentials` hides the secret values in the console log.
- **Docker socket and root user.** Mounting the Docker socket and running Jenkins as root give the Jenkins container full control of the host's Docker. This is convenient and acceptable on a personal machine, but a shared or production Jenkins should use dedicated build agents instead.
- **Screenshots.** The AWS account ID, the access key ID and the initial admin password were hidden in the screenshots published in the repository.

9. Advantages, Limitations and Future Scope

9.1 Advantages

- Every code change goes through the same automatic and repeatable steps, which removes manual errors.
- Tests run before deployment, so defects are found early.
- Infrastructure is described in code, so it can be reviewed, versioned, recreated and destroyed on demand.
- The whole setup can be reproduced on another computer from the repository and Docker alone.
- A failed stage immediately shows where the problem is, as the failure of build #1 showed.

9.2 Limitations

- Jenkins is started manually and builds are triggered by hand (**Build Now**); there is no webhook from GitHub yet.
- The Terraform state is stored locally in the Jenkins workspace, so it is lost if the workspace or volume is deleted and it cannot be shared by a team.
- The AWS and Docker deployments are separate: the application runs in a local container, not on the EC2 instance.
- There is one environment only and the application is very small.
- The security group allows HTTP from anywhere and no SSH access, which is simple but not a production design.

9.3 Future Scope

- **Automatic triggers:** add a GitHub webhook or SCM polling so each push starts a build.
- **Remote state:** store the Terraform state in an S3 bucket with a DynamoDB lock table, so the state survives and can be shared safely.
- **Deploy to EC2:** push the image to a registry (Docker Hub or Amazon ECR) and run it on the provisioned EC2 instance.
- **Multiple environments:** separate dev, test and production using Terraform workspaces or variable files, with a manual approval step before production.

- **Quality gates:** add linting, code coverage and security scanning of the image and the Terraform code.
- **Notifications:** send build results to e-mail or a chat tool.
- **Provider caching:** use a Terraform plugin cache directory to avoid repeated large downloads.

10. Conclusion

The project achieved its goal of building a working CI/CD workflow with Infrastructure as Code. A Jenkins pipeline defined in a Jenkinsfile checks out code from GitHub, installs dependencies, runs tests, builds a Docker image, validates and plans the AWS infrastructure, deploys the application container with Terraform and verifies it with a health check. A real `terraform apply` created an S3 bucket, a security group and an EC2 instance in AWS, and a code change pushed to GitHub was rebuilt and redeployed automatically.

Beyond the working result, the project showed why automation matters. A missing tool was found and located by the pipeline within seconds; a complete loss of the Jenkins configuration was repaired in minutes because the pipeline and infrastructure live in version control; and cloud resources could be created and removed with single commands. The practical lessons were to keep everything in code, protect secrets, review plans before applying, verify every deployment, and clean up cloud resources after use.

The project provides a solid base for a larger DevOps pipeline, and the future scope in Section 9.3 lists the natural next steps.

References

[1] Jenkins Project, "Jenkins User Documentation: Pipeline," https://www.jenkins.io/doc/book/pipeline/ (accessed November 2026).

[2] Jenkins Project, "Using a Jenkinsfile," https://www.jenkins.io/doc/book/pipeline/jenkinsfile/ (accessed November 2026).

[3] HashiCorp, "Terraform Documentation," https://developer.hashicorp.com/terraform/docs (accessed November 2026).

[4] HashiCorp, "Terraform AWS Provider," https://registry.terraform.io/providers/hashicorp/aws/latest/docs (accessed November 2026).

[5] Kreuzwerker, "Terraform Docker Provider," https://registry.terraform.io/providers/kreuzwerker/docker/latest/docs (accessed November 2026).

[6] Docker Inc., "Docker Documentation," https://docs.docker.com/ (accessed November 2026).

[7] Amazon Web Services, "Amazon S3 User Guide" and "Amazon EC2 User Guide," https://docs.aws.amazon.com/ (accessed November 2026).

[8] Node.js Foundation, "Node.js Test Runner," https://nodejs.org/api/test.html (accessed November 2026).

[9] H. Gujar, "jenkins-terraform-mini-project," GitHub repository, https://github.com/Harshadagujar2005/jenkins-terraform-mini-project.

Appendix A: Jenkinsfile

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = 'mini-app'
        IMAGE_TAG  = "${env.BUILD_NUMBER}"
        APP_PORT   = '3000'
    }

    parameters {
        booleanParam(name: 'APPLY_AWS', defaultValue: false,
                     description: 'Tick only if you have an AWS account + Jenkins credential "aws-
creds". Otherwise AWS is plan-only (no account needed).')
    }

    options {
        timestamps()
        disableConcurrentBuilds()
    }

    stages {
        stage('Checkout') {
            steps {
                // Pulls the code from the Git repository configured in the Jenkins job
                checkout scm
            }
        }

        stage('Install Dependencies') {
            steps {
                sh 'node --version && npm --version'
                sh 'npm install'
            }
        }

        stage('Test') {
            steps {
                sh 'npm test'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .'
            }
        }

        // ---------- Terraform: AWS cloud infrastructure (plan works without an AWS account)
----------
        stage('Terraform AWS: Init & Validate') {
            steps {
                dir('terraform/aws') {
                    sh 'terraform init -input=false'
                    sh 'terraform fmt -recursive'
                    sh 'terraform validate'
                }
            }
        }

        stage('Terraform AWS: Plan') {
            steps {
                dir('terraform/aws') {
                    sh 'terraform plan -input=false -no-color -var="use_mock_credentials=true" |
tee aws-plan.txt'
                }
                archiveArtifacts artifacts: 'terraform/aws/aws-plan.txt', allowEmptyArchive: true
            }
        }

        stage('Terraform AWS: Apply (optional)') {
            when { expression { return params.APPLY_AWS == true } }
            steps {
                withCredentials([usernamePassword(credentialsId: 'aws-creds',
                                                  usernameVariable: 'AWS_ACCESS_KEY_ID',
                                                  passwordVariable: 'AWS_SECRET_ACCESS_KEY')]) {
                    dir('terraform/aws') {
```

```
                        sh 'terraform apply -auto-approve -input=false -
var="use_mock_credentials=false"'
                    }
                }
            }
        }

        // ---------- Terraform: local Docker deployment (real apply, no cloud needed) ----------
        stage('Terraform Docker: Init & Validate') {
            steps {
                dir('terraform/docker') {
                    sh 'terraform init -input=false'
                    sh 'terraform fmt -recursive'
                    sh 'terraform validate'
                }
            }
        }

        stage('Terraform Docker: Plan') {
            steps {
                dir('terraform/docker') {
                    sh 'terraform plan -input=false -var="image_name=${IMAGE_NAME}" -
var="image_tag=${IMAGE_TAG}" -var="external_port=${APP_PORT}"'
                }
            }
        }

        stage('Deploy (Terraform Apply)') {
            steps {
                dir('terraform/docker') {
                    sh 'terraform apply -auto-approve -input=false -var="image_name=${IMAGE_NAME}"
-var="image_tag=${IMAGE_TAG}" -var="external_port=${APP_PORT}"'
                }
            }
        }

        stage('Verify Deployment') {
            steps {
                // host.docker.internal = the machine running Docker (where the app container
publishes its port)
                sh '''
                    for i in 1 2 3 4 5 6 7 8 9 10; do
                      if curl -fsS http://host.docker.internal:${APP_PORT}/health; then
                        echo; echo "Deployment verified."; exit 0
                      fi
                      echo "Waiting for app..."; sleep 3
                    done
                    echo "App did not become healthy"; exit 1
                '''
            }
        }
    }

    post {
        success { echo 'Pipeline succeeded. App is running at http://localhost:3000' }
        failure { echo 'Pipeline failed. Check the stage logs above.' }
    }
}
```

Appendix B: Terraform Configuration

terraform/aws/main.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# When use_mock_credentials = true (default), no real AWS account is needed:
# `terraform init`, `validate` and `plan` work offline with dummy keys.
# Set it to false (and provide real credentials) to run a real `apply`.
provider "aws" {
  region = var.aws_region

  access_key = var.use_mock_credentials ? "mock-access-key" : null
  secret_key = var.use_mock_credentials ? "mock-secret-key" : null

  skip_credentials_validation = var.use_mock_credentials
  skip_requesting_account_id  = var.use_mock_credentials
  skip_metadata_api_check     = var.use_mock_credentials
  skip_region_validation      = var.use_mock_credentials
}

# 1. S3 bucket to store build artifacts
resource "aws_s3_bucket" "artifacts" {
  bucket = var.bucket_name

  tags = {
    Name    = "${var.project_name}-artifacts"
    Project = var.project_name
  }
}

resource "aws_s3_bucket_versioning" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id

  versioning_configuration {
    status = "Enabled"
  }
}

# 2. Security group allowing web traffic
resource "aws_security_group" "web" {
  name        = "${var.project_name}-web-sg"
  description = "Allow HTTP inbound and all outbound"

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Project = var.project_name
  }
}

# 3. EC2 web server (free-tier instance type)
resource "aws_instance" "web" {
  ami                    = var.ami_id
  instance_type          = var.instance_type
```

```
  vpc_security_group_ids = [aws_security_group.web.id]

  user_data = <<-EOT
    #!/bin/bash
    yum install -y docker
    systemctl enable --now docker
    docker run -d -p 80:80 nginx
  EOT

  tags = {
    Name    = "${var.project_name}-web"
    Project = var.project_name
  }
}
```

terraform/aws/variables.tf

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "ap-south-1"
}

variable "use_mock_credentials" {
  description = "true = dummy credentials (plan only, no AWS account needed). false = use real
credentials."
  type        = bool
  default     = true
}

variable "project_name" {
  description = "Prefix used for resource names and tags"
  type        = string
  default     = "mini-cicd"
}

variable "bucket_name" {
  description = "Globally unique S3 bucket name (change this before a real apply)"
  type        = string
  default     = "mini-cicd-project-harshada"
}

variable "ami_id" {
  description = "Amazon Linux AMI ID for your region (look it up in the EC2 console before a real
apply)"
  type        = string
  default     = "ami-05fbd65e334e88c16"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}
```

terraform/aws/outputs.tf

```
output "bucket_name" {
  description = "S3 artifact bucket"
  value       = aws_s3_bucket.artifacts.bucket
}

output "security_group_id" {
  description = "Security group ID"
  value       = aws_security_group.web.id
}

output "instance_type" {
  description = "EC2 instance type used"
  value       = aws_instance.web.instance_type
}
```

terraform/docker/main.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {
  host = var.docker_host
}

# Reusable module that provisions one application container
module "app" {
  source = "./modules/app_container"

  container_name = var.container_name
  image          = "${var.image_name}:${var.image_tag}"
  internal_port  = 3000
  external_port  = var.external_port
  environment    = var.environment
}
```

terraform/docker/variables.tf

```
variable "docker_host" {
  description = "Docker daemon socket"
  type        = string
  default     = "unix:///var/run/docker.sock"
}

variable "container_name" {
  description = "Name of the container to create"
  type        = string
  default     = "mini-app"
}

variable "image_name" {
  description = "Docker image name built by Jenkins"
  type        = string
  default     = "mini-app"
}

variable "image_tag" {
  description = "Docker image tag (Jenkins build number)"
  type        = string
  default     = "latest"
}

variable "external_port" {
  description = "Port exposed on the host machine"
  type        = number
  default     = 3000
}

variable "environment" {
  description = "Environment label shown by the app"
  type        = string
  default     = "dev"
}
```

terraform/docker/outputs.tf

```
output "container_name" {
  description = "Name of the deployed container"
  value       = module.app.container_name
}

output "container_id" {
  description = "ID of the deployed container"
  value       = module.app.container_id
}

output "app_url" {
  description = "URL to open the application"
  value       = "http://localhost:${var.external_port}"
}
```

terraform/docker/modules/app_container/main.tf

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

resource "docker_container" "app" {
  name    = var.container_name
  image   = var.image
  restart = "unless-stopped"

  ports {
    internal = var.internal_port
    external = var.external_port
  }

  env = [
    "APP_ENV=${var.environment}",
    "PORT=${var.internal_port}"
  ]
}
```

terraform/docker/modules/app_container/variables.tf

```
variable "container_name" {
  type = string
}

variable "image" {
  type = string
}

variable "internal_port" {
  type    = number
  default = 3000
}

variable "external_port" {
  type = number
}

variable "environment" {
  type    = string
  default = "dev"
}
```

Appendix C: Steps to Reproduce the Project

1. Install Docker Desktop and Git; check with `docker --version` and `git --version`.
2. Clone the repository: `git clone https://github.com/Harshadagujar2005/jenkins-terraform-mini-project.git`.
3. For the real AWS part: create an IAM user with `AmazonS3FullAccess` and `AmazonEC2FullAccess`, create an access key, and note an Amazon Linux 2023 AMI ID for ap-south-1. Set a unique `bucket_name` and the `ami_id` in `terraform/aws/variables.tf` and push the change.
4. Start Jenkins: `cd jenkins` then `docker compose up -d --build` (the first build takes 5 to 10 minutes).
5. Get the unlock password with `docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword`, open http://localhost:8080, install the suggested plugins and create the admin user.
6. Add the credential: Manage Jenkins, Credentials, System, Global credentials, Add Credentials; kind Username with password; ID `aws-creds`; username = access key ID; password = secret access key.
7. Create a Pipeline job `mini-cicd-pipeline`: Definition = Pipeline script from SCM, SCM = Git, the repository URL, branch `*/main`, script path `Jenkinsfile`.
8. Click Build with Parameters and leave `APPLY_AWS` unticked for the safe first run. When it is green, open http://localhost:3000.
9. Run again with `APPLY_AWS` ticked to create the AWS resources; confirm them in the AWS console (Mumbai).
10. Change `server.js`, then `git add .`, `git commit -m "Change message"`, `git push`, and build again to see the automatic redeploy.
11. Clean up: inside the Jenkins container run `terraform destroy -var="use_mock_credentials=false"` in the `terraform/aws` workspace folder with the AWS keys exported, confirm in the console that the resources are gone, and deactivate the IAM access key.